TinyIDS: training run 2 (without `IAT`)

Removing the suspicious feature, a 20% sample, and RQ1 for two models

Run date: 06 November 2026, laptop CPU (16 cores). Took 20 minutes. Command: `python train_tinyids.py --sample 0.2 --drop IAT --run-name run2`

Contents

1 Why run 2? **2**

2 What changed from run 1 **2**

3 Results (test set, 1,176,851 rows) **3**

4 Run 1 vs run 2: the effect of removing `IAT` **3**

5 Does the 20% sample change the scores? **5**

6 SHAP feature ranking without `IAT` **5**

7 RQ1: how few features are enough? **6**

8 What is good **6**

9 What is not good, and the plan **7**

10 Next steps **7**

1 Why run 2?

Run 1 (all 46 features, full data) left three open questions:

1. **The `IAT` (Inter-Arrival Time) feature looked like a shortcut.** SHAP (SHapley Additive exPlanations) rated it about 5× more important than any other feature. Its values in CICIoT2023 look like capture timestamps rather than traffic behaviour, which a live IDS (Intrusion Detection System) cannot reproduce. **Question: how good are the models without it?**

2. **LightGBM (Light Gradient Boosting Machine) was oddly worse than Random Forest** on 8 and 34 classes, probably because its learning rate was too high.

3. **RQ1** (Research Question 1) found no feature subset within 0.02 of the full model, but only LightGBM was tested and only up to 20 features.

2 What changed from run 1

	Run 1	**Run 2**
Features	all 46	**45**: `IAT` left out (new `--drop` option)
Training rows	5,491,971 (100%)	**1,098,395 (20%)**, stratified: every attack keeps 20% of its rows
Test rows	1,176,851 (full)	1,176,851 (full, unchanged)
LightGBM	learning rate 0.1, max 400 rounds	**learning rate 0.05**, max 1000 rounds, early stop after 30
RQ1	LightGBM only; top 5/10/15/20	**LightGBM and Random Forest**; top 5/10/15/20/25/30
Memory	peaked at 97% of 15 GB	**about 3.6 GB**: data tables freed after loading, NN (Neural Network) preprocessing in chunks
Output folders	`results/`, `models/`	`results_run2/`, `models_run2/` (run 1 kept)
Time	32 min	**20 min**

Why 20%? A full-data run with the slower learning rate was started first, but was going to take until about 10 PM, so it was stopped after 7 models. Its partial results (`training_run2_full_STOPPED.log`) are used in section 5 to check how much the 20% sample changes the scores.

3 Results (test set, 1,176,851 rows)

Task	Model	Accuracy	Macro-F1	Benign FPR	Size (KB)	ms/row
binary	RF	0.9930	0.9322	0.0164	7,292	3.56
binary	**LightGBM**	**0.9945**	**0.9450**	0.0346	6,667	0.77
binary	MLP	0.9909	0.9158	0.0161	60	0.08
binary	1D-CNN	0.9858	0.8792	0.0139	83	0.22
8-class	RF	0.9052	0.6876	0.0806	39,464	3.66
8-class	**LightGBM**	**0.9099**	**0.7049**	0.0720	26,916	1.27
8-class	MLP	0.8724	0.6386	0.0868	62	0.09
8-class	1D-CNN	0.8628	0.6111	0.1335	84	0.50
34-class	**RF**	**0.8873**	**0.6472**	0.0721	54,321	3.88
34-class	LightGBM	0.8863	0.6266	0.1326	9,493	0.79
34-class	MLP	0.8404	0.5534	0.1067	69	0.09
34-class	1D-CNN	0.8301	0.5320	0.1272	91	0.18

FPR = False Positive Rate (share of normal traffic flagged as attack). MLP = Multi-Layer Perceptron; 1D-CNN = one-dimensional Convolutional Neural Network; RF = Random Forest.

4 Run 1 vs run 2: the effect of removing `IAT`

Best model of each task:

		Run 1 (with IAT)		**Run 2 (without IAT)**	
Task	**Model**	Accuracy	Macro-F1	Accuracy	Macro-F1
binary	LightGBM	0.9969	0.968	0.9945	0.945
8-class	RF / LightGBM	0.9955	0.864	0.9099	0.705
34-class	RF	0.9929	0.796	0.8873	0.647

Which classes suffered? (8-class, F1 per class)

Class	Run 1 RF	Run 2 LightGBM	
Mirai	0.999	0.997	unchanged
DDoS	1.000	0.944	small drop
Benign	0.943	0.898	small drop
Spoofing	0.874	0.777	drop
DoS	1.000	**0.748**	**big drop**
Recon	0.832	0.725	drop
BruteForce	0.713	**0.362**	**big drop**
Web	0.549	**0.188**	**big drop**

What this means

- **"Attack or not" does not depend on `IAT`.** Binary accuracy only went from 99.69% to 99.45%; attack-class F1 is still 0.997. This is the most important result for the live IDS.
- **DDoS vs DoS now get confused.** The only real difference between them is *how many machines* attack; a window of 10–100 packets looks almost the same either way. With `IAT` the model separated them perfectly; without it, it cannot. This strongly suggests `IAT` identified *which capture session* a row came from rather than how the attack behaves, i.e. a **dataset shortcut**.
- **Web and BruteForce collapse** (F1 0.19 and 0.36): with few rows and no `IAT`, their packet statistics look like normal or Recon traffic.
- **Conclusion:** run 1's 99.5% 8-class accuracy was **optimistic**. Run 2 is the honest number for an IDS that must work on live traffic, where `IAT` cannot be reproduced. This is a worthwhile finding for the report.

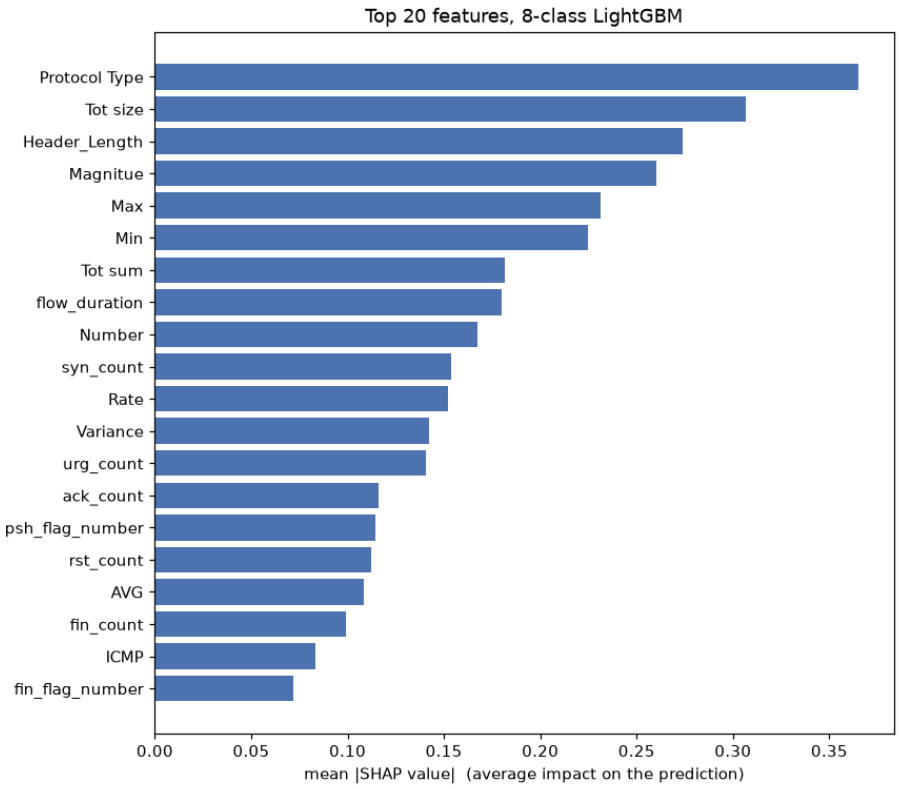
5 Does the 20% sample change the scores?

Comparison with the stopped full-data run (same settings, without `IAT`, 7 models had finished):

	100% data		20% data	
Model	Accuracy	Macro-F1	Accuracy	Macro-F1
binary RF	0.9923	0.927	0.9930	0.932
binary LightGBM	0.9944	0.945	0.9945	0.945
binary MLP	0.9917	0.923	0.9909	0.916
binary 1D-CNN	0.9897	0.907	0.9858	0.879
8-class RF	0.9055	0.720	0.9052	0.688
8-class LightGBM	0.9122	0.749	0.9099	0.705
8-class MLP	0.8976	0.668	0.8724	0.639

- **Binary and accuracy: practically no difference.** 20% is enough.
- **8-class macro-F1 is 0.03–0.04 lower with 20%**: the rare classes (Web, BruteForce) have only about 60–600 training rows left.
- **Plan:** experiment with 20% (fast); do one final full-data run of the chosen setup for the report numbers.

6 SHAP feature ranking without `IAT`

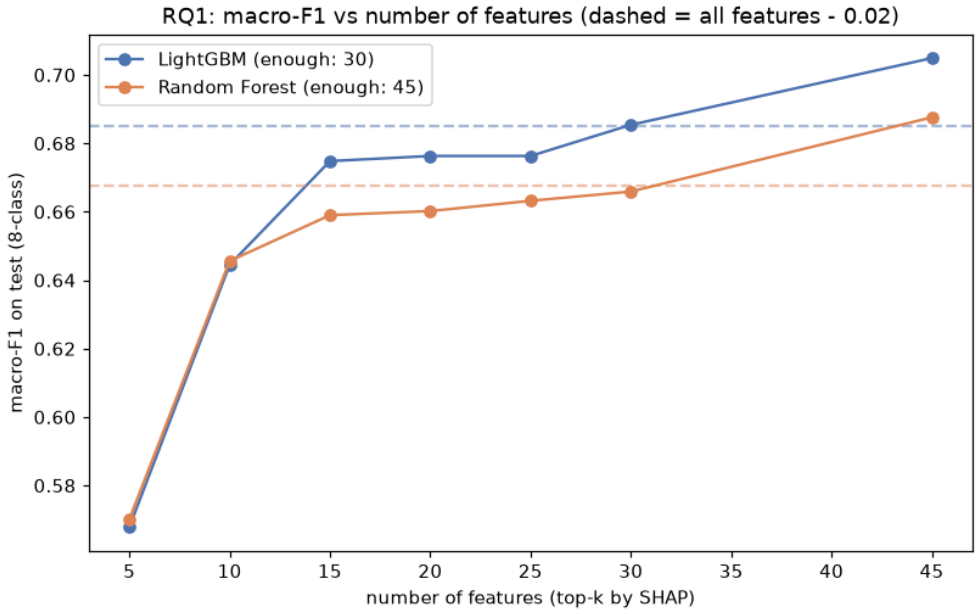


With `IAT` gone, importance is spread much more evenly (top feature 0.37 instead of 1.31). The top features are exactly what a network IDS should use: **protocol** (`Protocol Type`), **packet sizes** (`Tot size`, `Max`, `Min`, `Magnitue`), **header length**, **flow duration**, **rate** and **TCP flag counts** (`syn_count`, `ack_count`, `rst_count`). All of these can be computed from live packets on the Pi.

7 RQ1: how few features are enough?

8-class models retrained with the top-k features (ranking above), scored on the full test set:

	LightGBM		Random Forest	
Features	Macro-F1	Gap	Macro-F1	Gap
5	0.568	−0.137	0.570	−0.118
10	0.645	−0.060	0.646	−0.042
15	0.675	−0.030	0.659	−0.029
20	0.676	−0.029	0.660	−0.027
25	0.676	−0.029	0.663	−0.024
30	**0.685**	**−0.020**	0.666	−0.022
45 (all)	0.705	—	0.688	—



Answer from run 2:

- **LightGBM: 30 features** keep macro-F1 within 0.02 of all 45 (just on the line).
- **Random Forest:** 30 features are 0.022 short; no subset up to 30 is within 0.02.
- The curve rises steeply up to **15 features**, then flattens: from 15 to 30 features adds only about 0.01. **15 features is the sweet spot** for the Pi (fewer features to compute live, smaller model) at a cost of about 0.03 macro-F1.

8 What is good

- **Binary detection stays excellent without** `IAT`: 99.45% accuracy, attack-class F1 0.997. "Is this an attack?" works.
- **Mirai and DDoS floods are still detected very well** (F1 0.997 and 0.944): these cover the hping3 floods of the live demo.
- **The SHAP ranking now makes sense** and every top feature can be computed on the Pi.

- **LightGBM is now the best model** for binary and 8-class (the lower learning rate fixed run 1's problem), and its latency is 0.8–1.3 ms per row.
- **RQ1 has a clear answer for LightGBM** (30 features), and a practical sweet spot (15).
- **Memory is fixed** (3.6 GB instead of 15 GB) and a run takes 20 minutes.

9 What is not good, and the plan

Problem	**Plan**
8-class / 34-class scores dropped a lot without `IAT` (8-class macro-F1 0.86 → 0.70).	This is the honest result. For the report: show both runs and explain the shortcut. For the live IDS, use the model *without* `IAT`.
DDoS and DoS are confused (DoS F1 0.75).	Expected: they differ only in the number of attackers. Option: merge them into one "flood" class (7 classes), which is also more useful in practice.
Web (F1 0.19) and BruteForce (0.36) are almost undetectable.	Too few rows and similar-looking traffic. Option: stronger class weights or oversampling for these two; or report them as a known limitation.
Models are still far too big (7–54 MB vs < 1 MB).	Compression step (RQ2): fewer and shallower trees, top-15 features.
Benign false-alarm rate 7–13% for 8/34-class.	Too many normal rows flagged. Will improve with the flood-class merge and tuning; for the live IDS the binary model (1.6–3.5% FPR) can decide "attack or not" first.
20% sample costs about 0.03–0.04 macro-F1 on 8-class.	Final report numbers from one full-data run.

10 Next steps

1. Decide the final class setup (e.g. merge DDoS + DoS) and features (top 15 without `IAT`).
2. Compression (RQ2): small LightGBM / RF with top-15 features, export to ONNX (Open Neural Network Exchange); measure size and speed.
3. Live feature extractor + IDS on the laptop testbed, then on the Pi.
4. Final full-data training run of the chosen setup.